Generative classifiers are just language models: some simple tricks

Jialin Lu 2026-06-08

A generative classifier is a language model fine-tuned to write down a label. It is a good way to build a classifier and a slow way to run one. Qwen3Guard-Gen-0.6B, run the way its model card shows, takes about two seconds per call on a CPU. Three small changes bring that to about 250 milliseconds, and the labels do not change. None of the three is new: they are the usual tricks for serving a language model, and they apply here because a generative classifier is one.

Contents

We want to check text for safety: read each message and label it safe, unsafe, or controversial. It runs on every message, so it has to be cheap. A common way to build it today is to take a language model and fine-tune it to answer with the label. That works well but runs slowly: the model writes a whole sentence to tell us one word.

This post is about getting the word without the sentence. We will say exactly what such a model computes, find the work that does not change the answer, and remove it. If you serve language models already, the fixes will look familiar. The running example is Qwen3Guard-Gen-0.6B[1] on a CPU, the realistic place to run a sub-billion-parameter model at batch one. There the model-card path takes about two seconds per call, and the fast path takes about 250 milliseconds.

[1] Zhao, Haiyang, and others. 2025. "Qwen3guard Technical Report."

Two ways to build a classifier

A classifier takes an input and returns one of K labels; here $K = 3$. There are two ways to build one.

The standard way is a small head on top of an encoder. The encoder turns the input x into a vector $h(x) \in \mathbb{R}^d$. A linear layer of K units turns that into K scores, and a softmax turns the scores into a distribution over the labels,

$$p(y = k \texttt{ mid } x) = \frac{\exp\big(w_k^\top h(x)\big)}{\sum_{j=1}^{K} \exp\big(w_j^\top h(x)\big)}, \quad W \in \mathbb{R}^{K \times d}. \tag{1}$$

You train W and the encoder on labelled data with cross-entropy, and that is the whole model. This is a BERT-style text classifier: small, fast, and for many tasks the right choice.[2]

The second way, common lately for content safety, is to take a pretrained, instruction-tuned language model and fine-tune it to write the label as text. Llama Guard[3], ShieldGemma[4], and Qwen3Guard[5] all do this. You give the model a chat prompt and it generates

```
Safety: Unsafe
Categories: Violent
```

and you read the word after `Safety:`. I will call this a *generative classifier*, to set it apart from the head above.

Running a few-hundred-million-parameter model to choose among three labels is a lot of computation, so why do it this way? Because the pretrained model already knows a lot about language and the world, in many languages. A head trained from scratch knows only the labels it was shown. The generative classifier also lets you keep the policy in the prompt: the list of what counts as unsafe lives in the system prompt, not the weights, so you change it by editing text instead of retraining.[6] So the generative classifier is often the right tradeoff. As usually run, it is just not fast. To fix that, we first say exactly what it computes.

[2] Meta Llama Team. 2024. "Llama-Prompt-Guard-2-86m Model Card."

[3] Meta Llama Team. 2024. "Llama Guard 3-8b Model Card."

[4] Zeng, Wenjun, and others. 2024. "Shield-gemma: Generative AI Content Moderation Based on Gemma."

[5] Zhao, Haiyang, and others. 2025. "Qwen3guard Technical Report."

Listing 1: The model card's output format. The label is the word after `Safety:`.

[6] Alibaba ship a strict and a loose Qwen3Guard from the same weights and different system wording. A from-scratch classifier cannot do that; changing its label set means training a new one.

What the model computes

A language model does not have the head from Equation 1. It has a *language-modeling head*, the same kind of linear layer as W, only much wider. At each position t it projects the hidden state onto one score per vocabulary token, giving a distribution over the whole vocabulary $\mathcal{V}$ for the next token,

$$p(x_{t+1} = v \texttt{ mid } x_{\le t}) = \frac{\exp(E_v^\top h_t)}{\sum_{u \in \mathcal{V}} \exp(E_u^\top h_t)}, \quad E \in \mathbb{R}^{|\mathcal{V}| \times d}. \tag{2}$$

Here h_t is the hidden state at position t, and E is that wide projection, one row per vocabulary token. The vocabulary is large: for Qwen3 it is about 150,000 tokens. That is the wrong shape for a classifier, a distribution over 150,000 tokens instead of over $K = 3$ labels.

But the labels are tokens too. `safe`, `unsafe`, and `controversial` are each a single token; call their ids i_1, i_2, i_3. End the prompt so that the next token the model would write is the verdict, by ending it with `Safety:` (more on that below).

Then read Equation 2 at that one position, keep only those three logits, and renormalize:

$$p(y = k \texttt{ mid } x) = \frac{\exp\left(E_{i_k}^{\top} h_t\right)}{\sum_{j=1}^{K} \exp\left(E_{i_j}^{\top} h_t\right)}. \tag{3}$$

Put Equation 3 next to Equation 1 and they are the same computation. We did not add a classification head; the head was already there, as the three rows $E_{i_1}, E_{i_2}, E_{i_3}$ of the language-modeling head. We just read those three rows, at one position.

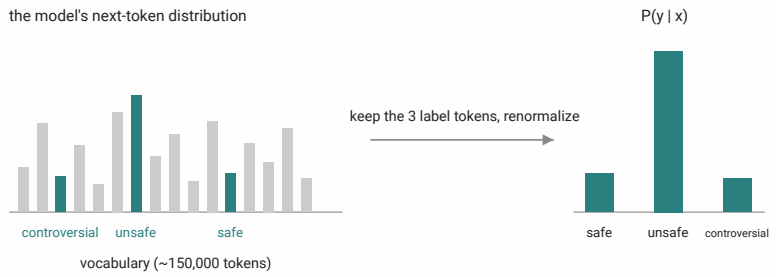


Figure 1: Classification as a distribution: pick the three label tokens out of the next-token distribution over the whole vocabulary, and renormalize. The classification head is three rows of the language-modeling head.

The chat template and the literal `Safety:` are there only to put the model in the state where its next token is the verdict. They are not part of the answer. So the whole job is one read:

> *Classification reads the next-token distribution at one position, over K tokens, after a fixed prompt.*

Anything the model does beyond that read does not change the label.

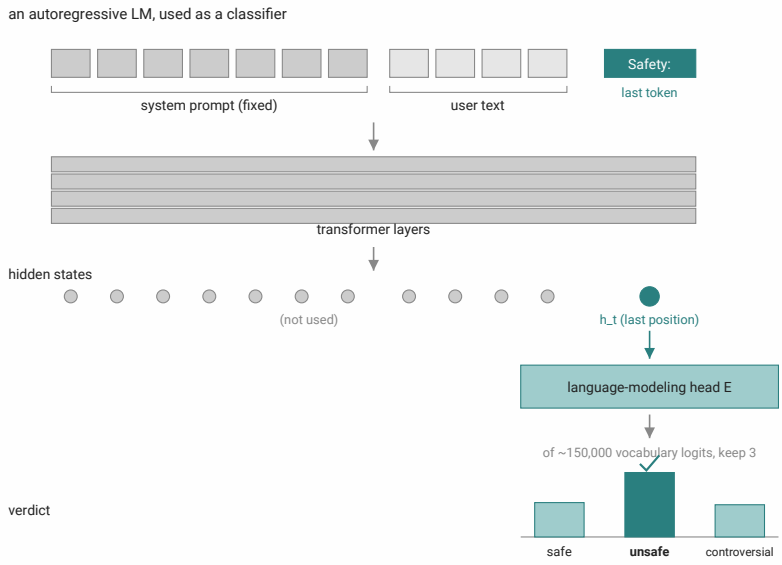


Figure 2: An autoregressive language model used as a classifier: the whole prompt runs through, but the verdict is one read at the last position.

Where the time goes

We need one read. The model-card path does much more, and the gap is where the time goes.

That path renders the chat prompt, calls `generate()`, and parses the text it gets back. The call is a *prefill* pass over the N prompt tokens, then a *decode loop* that writes the output one token at a time, each token its own forward pass. Qwen3Guard writes about nine tokens before the line we want is done,[7] so the decode loop runs about nine times. We pay for about ten forward passes, one prefill and nine decode, when we need one. Each forward pass on a small model does little math but carries a fixed overhead, so nine in a row is most of the time. On a CPU that is the gap between two seconds and a fifth of a second.

[7] `Safety: Unsafe`, then the `Categories:` line it was fine-tuned to always add.

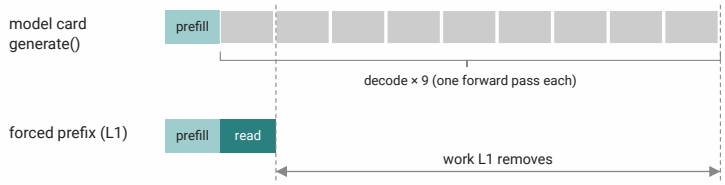


Figure 3: One call under the model-card recipe: a prefill pass and nine decode steps. The label is fixed by the end of the first step, so the rest is unused.

Line the path up against the one read, and three pieces of unused work fall out. We need the distribution at one position; the decode loop computes nine. We need three logits; the head projects every position onto all 150,000 tokens. We send the same fixed prompt every call; the model re-reads it from scratch each time. Three tricks remove them, one each.

Three tricks

Here is the part that should look familiar. Stopping a decode early, reading the logits at one position, reusing a fixed prefix's cache: these are everyday moves for serving a language model. A generative classifier is a language model being served, so it gets all three with no extra work. A team that treats the safety model as a black box, like an older classifier, pays for the full generation anyway, which on this model is about five times the cost it needs.

L1: stop decoding

We read one position, so generating the rest of the string is wasted.

The model was fine-tuned to always start its answer with `Safety:` , so writing that prefix ourselves changes nothing after it. It just skips to the token that varies. We append `Safety:` to the prompt, run one forward pass, and read the next-token distribution at the last position. The argmax over the three label logits is the verdict. This is *forced decoding*: we hand the model the tokens it would have written instead of letting it write them. ShieldGemma's card describes the same

procedure,[8] and Llama Guard describes the first-token-logit version.[9] No decode loop: one forward pass instead of about ten. This is the largest saving of the three, because it removes nine of the ten passes.

L2: only the last position

We read one position, so projecting the others onto the vocabulary is wasted.

The head in Equation 2 is a matrix multiply of shape $[N, d] \times [d, |\mathcal{V}|]$: project each of the N positions onto 150,000 logits. Equation 3 reads only the last position, so we slice the hidden states from $[N, d]$ to $[1, d]$ before the projection and compute that one row. The other $N - 1$ projections never happen.[10]

L3: reuse the fixed prompt

The system prompt, the chat template plus the policy text, is identical on every call, so its work can be reused.

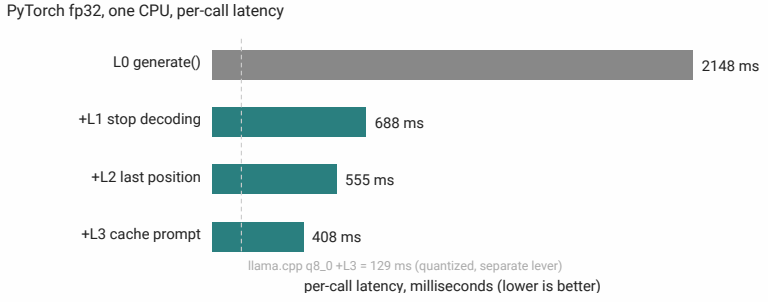
As the model reads the prompt, each token produces key and value vectors that later positions attend to; together these are the KV cache. The prompt prefix never changes, so its keys and values never change across calls, and we can compute them once and keep them.[11] After the first call, each call pays only for the user's text, not for re-reading the policy.

That is all three. Now the measurement.

Results

The first question is whether the answer survives, and it does. On every sample the fast path returns the same verdict as the model-card path, and at fp32 the logits match. So the rest is about speed.

The tricks change which numbers the model computes, not how each number is stored. So they hold on any backend and in any precision, and the savings are largest where each forward pass is expensive, which is the CPU. The model is Qwen3Guard-Gen-0.6B, batch one, on sixteen cores of a Kunpeng 920[12] server CPU, across three backends: PyTorch, ONNX Runtime,[13] and llama.cpp.[14] The input is a few hundred tokens, and each number is the median of 100 timed calls.



[8] Zeng, Wenjun, and others. 2024. "Shield-gemma: Generative AI Content Moderation Based on Gemma."

[9] Meta Llama Team. 2024. "Llama Guard 3-8b Model Card."

[10] In PyTorch this is `logits_to_keep=1`; in ONNX it is a slice node on the graph; llama.cpp and most runtimes already return only the last position. Narrowing the projection further, to just the three label rows, saves under a millisecond on this model, so we leave the head whole and slice only the position.

[11] In practice: cache the longest prefix that is the same regardless of user content, and run the forward pass only over the variable suffix.

[12] Huawei. n.d. "Kunpeng 920 Processor"

[13] Microsoft. 2018. "ONNX Runtime."

[14] Gerganov, Georgi, and the llama.cpp contributors. 2023. "Llama.cpp."

Figure 4: The ladder on one backend (PyTorch fp32). The decode loop, removed by L1, is most of the cost; L2 and L3 take the rest.

Table 1: Per-call latency, p50 ms, walking the ladder on three backends. L0 is the model-card `generate()` path, the same code the Qwen3Guard-Gen card shows. llama.cpp returns only the last position by default, so L2 is already in its baseline; that row is also 8-bit quantized, which is why it starts lower.

backend	L0 baseline	+L1	+L2	+L3
PyTorch fp32	2148	688	555	408
ONNX fp32	1671	598	485	253
llama.cpp q8_0	643	261	–	129

The shape is the same on every backend: L1 removes most of the time, then L2 and L3 take more off. The three tricks alone bring PyTorch from 2148 to 408 ms, about five times faster. Keep the tricks but switch to a faster fp32 runtime, and ONNX reaches 253 ms, **8.5 times faster** than the model-card reference, still at fp32 with identical verdicts.

Quantization, if you can spend the accuracy

Quantization is a separate lever from the three tricks, and it is the obvious next one. Storing the weights in 8 bits instead of 32 shrinks the model and speeds up every matrix multiply, on top of whatever tricks are already in place.

config (with the tricks)	p50 ms	vs. reference
ONNX fp32 (no quant)	253	8.5×
ONNX int8	164	13×
llama.cpp q8_0	129	17×

Table 2: Best fp32 path against two 8-bit paths, all with the tricks applied, p50 ms. The reference is the 2148 ms model-card path.

The win is real and so is the cost. The fp32 paths reproduce the reference verdict on every sample. The 8-bit paths do not: int8 agrees with fp32 on about 98 of 100 inputs, and the two it misses are borderline, near the safe/controversial line. [15] For most uses that is a fine trade. If your application cannot move a single borderline label, stay at fp32 and take the 8.5×.

[15] Weight-only 8-bit quantization, fp32 accumulation. The drift is a fraction of a logit, enough to flip a verdict only where the top two labels were already almost tied.

Summary

A generative classifier is a language model asked to write down a label, and that one fact is the whole post. Its classification head is three rows of the model's own vocabulary projection, read at one position after a fixed prompt; everything past that read is unused. The three tricks remove it: stop decoding, read only the last position, reuse the fixed prompt. They hold on any backend, in any precision, and leave the output unchanged. Together they take a two-second CPU call down to about 250 ms, 8.5 times faster, and quantization roughly halves it again.

None of this is special to safety. It holds whenever a generative model gives a short, fixed-shape answer: most of the generation is not part of the answer, and once you stop treating the model as a black box, the tricks to skip it are the ones you already use.

Appendix: GPU and the larger models

On a GPU, only L1 matters

On a GPU the forced-prefix trick (L1) is still the one that matters, because the decode loop's fixed per-step cost dominates there too. On an RTX 3090, Qwen3Guard-Gen-0.6B with L1 runs about 29 ms p50, against 237 ms for the model-card path, at a comparable input length. L2 and L3 do not help on the GPU, where the vocabulary projection and prompt re-reading are cheap next to per-call overhead. On CPU the opposite holds, which is why the full ladder matters there.

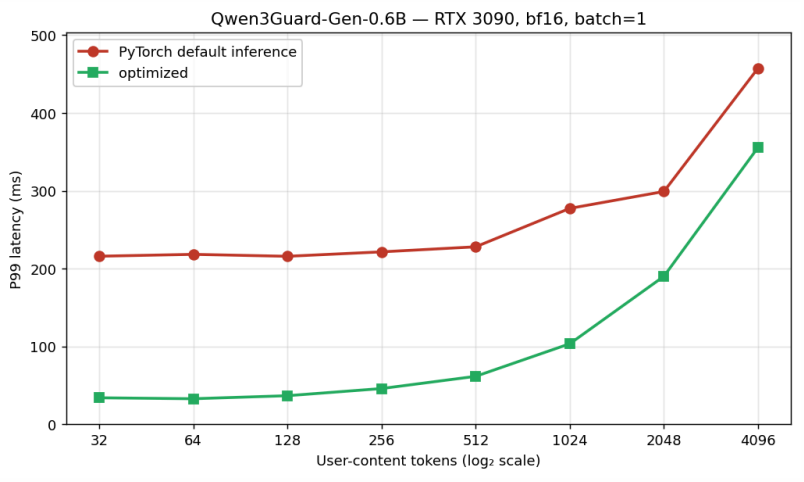


Figure 5: Qwen3Guard-Gen-0.6B on an RTX 3090: the model-card path against the forced-prefix path (L1), p99 latency across input lengths. The forced-prefix path stays under a 200 ms budget out to about 2048 input tokens.

Larger models in the family

The same construction works on the larger members of the family.

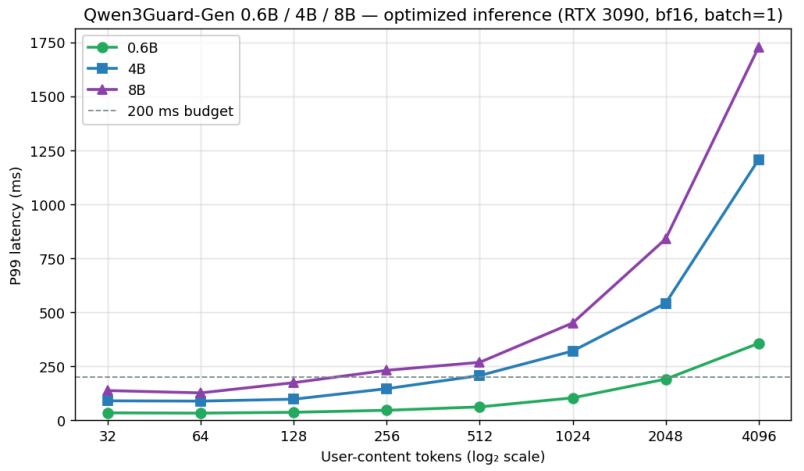


Figure 6: The forced-prefix path across the three Qwen3Guard-Gen sizes (0.6B / 4B / 8B) on an RTX 3090. All three get the same trick; the 0.6B clears a 200 ms budget at the longest inputs, the larger two hit it sooner.

Bibliography

Gerganov, Georgi, and the llama.cpp contributors. 2023. “Llama.cpp.”.

Huawei. n.d. “Kunpeng 920 Processor.”

Meta Llama Team. 2024. “Llama Guard 3-8b Model Card.”.

Meta Llama Team. 2024. “Llama-Prompt-Guard-2-86m Model Card.”.

Microsoft. 2018. “ONNX Runtime.”.

Zeng, Wenjun, and others. 2024. “Shieldgemma: Generative AI Content Moderation Based on Gemma.”.

Zhao, Haiyang, and others. 2025. “Qwen3guard Technical Report.”.